# Parallel & Distributed Computing

MPI Code Snippets

## Contents

# 1 Code Snippets

## 1.1 Code Snippet #1

**Run with 3 processes**

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{

    int rank, size;
    // Size = 3, in this case
    /* Start up MPI */
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);


    // MPI_SEND only copies value to system buf of the destination rank
    // Data send by one MPI_Send can only be received by one MPI_RECV
    // if two MPI_SEND calls are made to same rank, then 1 MPI_RECV will recv data
    // from the first mpi_send, no matter how much you increase the count
    // MPI_SEND and MPI_RECV work as a couple

    int *sendBuf = (int *)malloc(2 * sizeof(int));
    sendBuf[0] = 5;
    sendBuf[1] = 7;
    int recv[2] = {0, -1};

    if (rank == 0){
        MPI_Send(sendBuf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);        // Send # 1
        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);    // Send # 2
        printf("Both of the data send\n");
    }

    else if (rank == 1){
        sleep(5);
        // The recv count must be >= send count, otherwise it'll throw error
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        // Only recv data from send#1
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        // Only recv data from send#2
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }
    // MPI recv if the size is more than the data to be received then it ignores
    // (doesn't pick up junk)
    else if (rank == 2){
        // this one will be blocked for eternity
        MPI_Recv(&recv, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }


    MPI_Finalize();

    return 0;
}
```

### 1.2 Code Snippet #2

**Run with 2 processes**

```c

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{

    int rank, size;
    // Size = 2, in this case
    /* Start up MPI */
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);


    int *sendBuf = (int *)malloc(2 * sizeof(int));
    sendBuf[0] = 5;
    sendBuf[1] = 7;
    int recv[2] = {-1, -1};

    if (rank == 0){
        MPI_Send(sendBuf, 0, MPI_INT, 1, 0, MPI_COMM_WORLD);        // Send # 1
        printf("Both of the data send\n");
    }

    else if (rank == 1){
        sleep(5);
        MPI_Status status;
        int dataReceived;
        // The recv count must be >= send count, otherwise it'll throw error
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        MPI_Get_count(&status, MPI_INT, &dataReceived);
        printf("Count of data received = %d\n", dataReceived);
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }


    MPI_Finalize();

    return 0;
}
```

### 1.3 Code Snippet #3

**Run with 3 processes**

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *send_data = (int *)malloc(size * sizeof(int));
    int *recv_data = (int *)malloc(size * sizeof(int));

    // Initializing Data
    for (int i = 0; i < size; i++) {
        send_data[i] = i + rank + i * rank;
    }
```

```
    printf("Process %d : \n", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", send_data[i]);
    }
    printf("\n");

    // Array to store MPI_Request objects for non-blocking sends
    MPI_Request *send_requests = (MPI_Request *)malloc(size * sizeof(MPI_Request));

    // Send data to all processes (including itself)
    for (int i = 0; i < size; i++) {
        MPI_Isend(&send_data[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD, &send_requests[i]);
    }

    // Receive data from all processes (including itself)
    for (int i = 0; i < size; i++) {
        MPI_Recv(&recv_data[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    // Wait for all non-blocking sends to complete
    MPI_Waitall(size, send_requests, MPI_STATUSES_IGNORE);

    // Print the received data
    printf("Process %d received: ", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", recv_data[i]);
    }
    printf("\n");

    MPI_Finalize();
    return 0;
}
```

### 1.4 Code Snippet #4

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_TASKS 5
#define TASK_DATA_SIZE 3
#define TERMINATION_TAG 2

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        fprintf(stderr, "This program requires at least 2 processes.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (rank == 0) {
        // Master process
        int task_data[NUM_TASKS][TASK_DATA_SIZE] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9},
            {10, 11, 12},
            {13, 14, 15}
        };

        MPI_Request send_requests[NUM_TASKS];
```

```
        MPI_Request recv_requests[NUM_TASKS];
        int recv_buffer[NUM_TASKS][TASK_DATA_SIZE];
        int completed_tasks = 0;

        // Send tasks to workers
        for (int i = 0; i < NUM_TASKS; i++) {
            int worker_rank = (i % (size - 1)) + 1; // Distribute tasks among workers
            MPI_Isend(task_data[i], TASK_DATA_SIZE, MPI_INT, worker_rank, 0, MPI_COMM_WORLD,
                &send_requests[i]);
            printf("Master sent task %d to worker %d\n", i, worker_rank);
        }

        // Post nonblocking receives to get results from workers
        for (int i = 0; i < NUM_TASKS; i++) {
            MPI_Irecv(recv_buffer[i], TASK_DATA_SIZE, MPI_INT, MPI_ANY_SOURCE, 1,
                MPI_COMM_WORLD, &recv_requests[i]);
        }

        // Wait for all tasks to complete
        while (completed_tasks < NUM_TASKS) {
            int index;
            MPI_Status status;
            // wait if any of the requests is completed
            MPI_Waitany(NUM_TASKS, recv_requests, &index, &status);

            int count;
            MPI_Get_count(&status, MPI_INT, &count);

            printf("Master received result from worker %d: ", status.MPI_SOURCE);
            for (int j = 0; j < count; j++) {
                printf("%d ", recv_buffer[index][j]);
            }
            printf("\n");
            completed_tasks++;
        }

        // Wait for all send operations to complete
        MPI_Waitall(NUM_TASKS, send_requests, MPI_STATUSES_IGNORE);

        // Send a termination message to each worker
        for (int worker = 1; worker < size; worker++) {
            // Send an empty message with TERMINATION_TAG to indicate no more tasks.
            // See the working of send count = 0, how beneficial it is
            MPI_Send(NULL, 0, MPI_INT, worker, TERMINATION_TAG, MPI_COMM_WORLD);
            printf("Master sent termination message to worker %d\n", worker);
        }
    } else {
        // Worker processes
        MPI_Status status;
        int task_data[TASK_DATA_SIZE];
        int result_data[TASK_DATA_SIZE];

        while (1) {
            // Probe for incoming tasks or termination signal
            // Blocking call, only use if you know a send is called
            // Otherwise can use Irecv
            MPI_Probe(0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

            // Check if the incoming message is a termination message
            if (status.MPI_TAG == TERMINATION_TAG) {
                // Receive the termination message and break out of the loop.
                MPI_Recv(NULL, 0, MPI_INT, 0, TERMINATION_TAG, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
                printf("Worker %d received termination signal. Exiting.\n", rank);
                break;
            }

            int count;
            MPI_Get_count(&status, MPI_INT, &count);
            MPI_Recv(task_data, count, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
            printf("Worker %d received task: ", rank);
            for (int i = 0; i < count; i++) {
                printf("%d ", task_data[i]);
            }
            printf("\n");

            // Process the task (e.g., multiply each element by 2)
            for (int i = 0; i < count; i++) {
                result_data[i] = task_data[i] * 2;
            }

            // Send the result back to the master using nonblocking send.
            MPI_Request send_request;
            MPI_Isend(result_data, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &send_request);

            // Test if the send operation has completed
            int flag;
            MPI_Test(&send_request, &flag, MPI_STATUS_IGNORE);
            if (flag) {
                printf("Worker %d finished sending result.\n", rank);
            } else {
                MPI_Wait(&send_request, MPI_STATUS_IGNORE);
                printf("Worker %d finished sending result after waiting.\n", rank);
            }
        }
    }

    MPI_Finalize();
    return 0;
}
```

### 1.5 Code Snippet #5

**Run with 2 processes**

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *sendBuf = (int *)malloc(2 * sizeof(int));
    sendBuf[0] = 5;
    sendBuf[1] = 7;
    int recv[2] = {0, -1};

    if (rank == 0){
        MPI_Send(sendBuf, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Both of the data send\n");
    }
    else if (rank == 1){
        sleep(5);
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }

    MPI_Finalize();
    return 0;
}
```

### 1.6 Code Snippet #6

**Run with 2 processes**

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *sendBuf = (int *)malloc(2 * sizeof(int));
    sendBuf[0] = 5;
    sendBuf[1] = 7;
    int recv[2] = {0, -1};

    if (rank == 0){
        MPI_Send(sendBuf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Both of the data send\n");
    }
    else if (rank == 1){
        sleep(5);
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }

    MPI_Finalize();
    return 0;
}
```

### 1.7 Code Snippet #7

**Run with 2 processes**

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *sendBuf = (int *)malloc(2 * sizeof(int));
    sendBuf[0] = 5;
    sendBuf[1] = 7;
    int recv[2] = {0, -1};

    if (rank == 0){
        MPI_Send(sendBuf, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Both of the data send\n");
    }
    else if (rank == 1){
        sleep(5);
        MPI_Recv(recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
    }

```

```
    MPI_Finalize();
    return 0;
}
```

### 1.8 Code Snippet #8

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define ARRAY_SIZE 16

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = ARRAY_SIZE / size;
    int *data1 = (int *)malloc(ARRAY_SIZE * sizeof(int));
    int *data2 = (int *)malloc(ARRAY_SIZE * sizeof(int));
    int *sub_data1 = (int *)malloc(chunk_size * sizeof(int));
    int *sub_data2 = (int *)malloc(chunk_size * sizeof(int));

    // Root initializes two arrays
    if (rank == 0) {
        data1 = (int *)malloc(ARRAY_SIZE * sizeof(int));
        data2 = (int *)malloc(ARRAY_SIZE * sizeof(int));
        for (int i = 0; i < ARRAY_SIZE; i++) {
            data1[i] = i + 1;  // 1, 2, 3, ..., 16
            data2[i] = (i + 1) * 3; // 3, 6, 9, ..., 48
        }
        printf("Root initialized data1: ");
        for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data1[i]);
        printf("\n");

        printf("Root initialized data2: ");
        for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data2[i]);
        printf("\n");
    }

    MPI_Scatter(data1, chunk_size, MPI_INT, sub_data1, chunk_size, MPI_INT, 0,
        MPI_COMM_WORLD);

    int threshold = 10;
    MPI_Bcast(&threshold, 1, MPI_INT, 0, MPI_COMM_WORLD);

    for (int i = 0; i < chunk_size; i++) {
        if (sub_data1[i] > threshold) sub_data1[i] *= 2;
    }

    MPI_Gather(sub_data1, chunk_size, MPI_INT, data1, chunk_size, MPI_INT, 0, MPI_COMM_WORLD
        );

    if (rank == 0) {
        printf("Root gathered modified data1: ");
        for (int i = 0; i < ARRAY_SIZE; i++) printf("%d ", data1[i]);
        printf("\n");
    }

    MPI_Barrier(MPI_COMM_WORLD);
    MPI_Bcast(data1, ARRAY_SIZE, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Scatter(data2, chunk_size, MPI_INT, sub_data2, chunk_size, MPI_INT, 0,
        MPI_COMM_WORLD);

```

```
    int local_sum1 = 0, local_sum2 = 0;
    for (int i = 0; i < chunk_size; i++) {
        local_sum1 += sub_data1[i];
        local_sum2 += sub_data2[i];
    }

    int total_sum1 = 0, total_sum2 = 0;
    MPI_Reduce(&local_sum1, &total_sum1, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&local_sum2, &total_sum2, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    MPI_Bcast(&total_sum1, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&total_sum2, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int recv_val;
    MPI_Request send_request, recv_request;
    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    MPI_Isend(&local_sum1, 1, MPI_INT, next, 0, MPI_COMM_WORLD, &send_request);
    MPI_Irecv(&recv_val, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &recv_request);

    MPI_Wait(&send_request, MPI_STATUS_IGNORE);
    MPI_Wait(&recv_request, MPI_STATUS_IGNORE);

    printf("Process %d received value %d from process %d\n", rank, recv_val, prev);

    MPI_Finalize();
    return 0;
}
```

### 1.9 Code Snippet #9

**Run with 6 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define DATA_SIZE 12 // Total size of the dataset
#define CONFIG_VALUE 5 // Configuration parameter to broadcast

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Check if the number of processes divides the data size evenly
    if (DATA_SIZE % size != 0) {
        if (rank == 0) {
            fprintf(stderr, "DATA_SIZE must be divisible by the number of processes.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int local_size = DATA_SIZE / size;
    int *local_data = (int *)malloc(local_size * sizeof(int));
    int *global_data = NULL;
    int config_value;

    // Root process initializes the global dataset
    if (rank == 0) {
        global_data = (int *)malloc(DATA_SIZE * sizeof(int));
        for (int i = 0; i < DATA_SIZE; i++) {
            global_data[i] = i + 1; // Initialize with values 1, 2, 3, ..., DATA_SIZE
        }
    }

```

```
    MPI_Bcast(&config_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        config_value = CONFIG_VALUE;
    }
    MPI_Bcast(&config_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received config_value = %d\n", rank, config_value);

    MPI_Scatter(global_data, local_size, MPI_INT, local_data, local_size, MPI_INT, 0,
        MPI_COMM_WORLD);

    for (int i = 0; i < local_size; i++) {
        local_data[i] *= config_value;
    }

    MPI_Gather(local_data, local_size, MPI_INT, global_data, local_size, MPI_INT, 0,
        MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Final result gathered by root process:\n");
        for (int i = 0; i < DATA_SIZE; i++) {
            printf("%d ", global_data[i]);
        }
        printf("\n");
    }


    MPI_Finalize();
    return 0;
}
```

### 1.10 Code Snippet #10

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define ROOT 0

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *data = NULL, *recv_chunk = NULL, *send_counts = NULL, *displs = NULL;
    int *gather_displs = NULL, *gather_counts = NULL;
    int *alltoall_send = NULL, *alltoall_recv = NULL;

    int total_size = size * (size + 1) / 2;  // Uneven distribution formula
    int recv_size = rank + 1;  // Each process gets (rank + 1) elements

    if (rank == ROOT) {
        data = (int *)malloc(total_size * sizeof(int));
        send_counts = (int *)malloc(size * sizeof(int));
        displs = (int *)malloc(size * sizeof(int));

        int index = 0;
        for (int i = 0; i < size; i++) {
            send_counts[i] = i + 1;  // Process i gets i+1 elements
            displs[i] = index;
            index += send_counts[i];
        }

        for (int i = 0; i < total_size; i++) {
            data[i] = i + 1;  // Initialize with sequential values
        }
```

```
37
38          printf("Root initialized data: ");
39          for (int i = 0; i < total_size; i++) {
40              printf("%d ", data[i]);
41          }
42          printf("\n");
43      }
44
45      recv_chunk = (int *)malloc(recv_size * sizeof(int));
46
47      MPI_Scatterv(data, send_counts, displs, MPI_INT, recv_chunk, recv_size, MPI_INT, ROOT,
            MPI_COMM_WORLD);
48
49      // Barrier: Ensures all processes have received their chunks before proceeding
50      MPI_Barrier(MPI_COMM_WORLD);
51
52      for (int i = 0; i < recv_size; i++) {
53          recv_chunk[i] *= (rank + 2);
54      }
55
56      alltoall_send = (int *)malloc(size * sizeof(int));
57      alltoall_recv = (int *)malloc(size * sizeof(int));
58
59      for (int i = 0; i < size; i++) {
60          alltoall_send[i] = rank * 100 + i;  // Unique values per process
61      }
62
63      MPI_Alltoall(alltoall_send, 1, MPI_INT, alltoall_recv, 1, MPI_INT, MPI_COMM_WORLD);
64
65      MPI_Barrier(MPI_COMM_WORLD);
66
67      if (rank == ROOT) {
68          gather_counts = (int *)malloc(size * sizeof(int));
69          gather_displs = (int *)malloc(size * sizeof(int));
70
71          int index = 0;
72          for (int i = 0; i < size; i++) {
73              gather_counts[i] = i + 1;
74              gather_displs[i] = index;
75              index += gather_counts[i];
76          }
77      }
78
79      MPI_Gatherv(recv_chunk, recv_size, MPI_INT, data, gather_counts, gather_displs, MPI_INT,
             ROOT, MPI_COMM_WORLD);
80
81      if (rank == ROOT) {
82          printf("Root gathered modified data: ");
83          for (int i = 0; i < total_size; i++) {
84              printf("%d ", data[i]);
85          }
86          printf("\n");
87      }
88
89
90      MPI_Finalize();
91      return 0;
92  }
```

### 1.11 Code Snippet #11

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define DATA_SIZE 12 // Total size of the dataset

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (DATA_SIZE % size != 0) {
        if (rank == 0) {
            fprintf(stderr, "DATA_SIZE must be divisible by the number of processes.\n");
        }
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int local_size = DATA_SIZE / size;
    int *local_data = (int *)malloc(local_size * sizeof(int));
    int *global_data = NULL;
    int *send_counts = NULL;
    int *displs = NULL;

    if (rank == 0) {
        global_data = (int *)malloc(DATA_SIZE * sizeof(int));
        send_counts = (int *)malloc(size * sizeof(int));
        displs = (int *)malloc(size * sizeof(int));

        // Initialize global dataset with values 1, 2, 3, ..., DATA_SIZE
        for (int i = 0; i < DATA_SIZE; i++) {
            global_data[i] = i + 1;
        }

        for (int i = 0; i < size; i++) {
            send_counts[i] = local_size;
            displs[i] = i * local_size;
        }
    }

    MPI_Scatterv(global_data, send_counts, displs, MPI_INT, local_data, local_size, MPI_INT,
        0, MPI_COMM_WORLD);

    for (int i = 0; i < local_size; i++) {
        local_data[i] *= (rank + 1);
    }

    MPI_Barrier(MPI_COMM_WORLD);

    MPI_Gatherv(local_data, local_size, MPI_INT, global_data, send_counts, displs, MPI_INT,
       0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Final result gathered by root process:\n");
        for (int i = 0; i < DATA_SIZE; i++) {
            printf("%d ", global_data[i]);
        }
        printf("\n");
    }

    int *send_buffer = (int *)malloc(size * sizeof(int));
    int *recv_buffer = (int *)malloc(size * sizeof(int));

    for (int i = 0; i < size; i++) {
```

```
        send_buffer[i] = rank + 1;
    }

    MPI_Alltoall(send_buffer, 1, MPI_INT, recv_buffer, 1, MPI_INT, MPI_COMM_WORLD);

    printf("Process %d received: ", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", recv_buffer[i]);
    }
    printf("\n");

    MPI_Finalize();
    return 0;
}
```

### 1.12 Code Snippet #12

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define ROOT 0

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *send_data = NULL, *recv_chunk = NULL;
    int *send_counts = NULL, *send_displs = NULL;
    int *recv_counts = NULL, *recv_displs = NULL;
    int *gather_counts = NULL, *gather_displs = NULL;

    int total_size = size * (size + 1) / 2;  // Uneven data distribution
    int recv_size = (rank + 1) * 2;  // Randomized recv size

    if (rank == ROOT) {
        send_data = (int *)malloc(total_size * sizeof(int));
        send_counts = (int *)malloc(size * sizeof(int));
        send_displs = (int *)malloc(size * sizeof(int));

        int index = 0;
        for (int i = 0; i < size; i++) {
            send_counts[i] = (i + 1) * 2;
            send_displs[i] = index;
            index += send_counts[i];
        }

        for (int i = 0; i < total_size; i++) {
            send_data[i] = i + 1;
        }

        printf("Root initialized data: ");
        for (int i = 0; i < total_size; i++) {
            printf("%d ", send_data[i]);
        }
        printf("\n");
    }

    recv_chunk = (int *)malloc(recv_size * sizeof(int));

    MPI_Scatterv(send_data, send_counts, send_displs, MPI_INT, recv_chunk, recv_size,
        MPI_INT, ROOT, MPI_COMM_WORLD);

    MPI_Barrier(MPI_COMM_WORLD);  // Sync before modifications
```

```
    for (int i = 0; i < recv_size; i++) {
        recv_chunk[i] *= (rank + 1) * 3;
    }

    int *alltoall_send = (int *)malloc(size * size * sizeof(int));
    int *alltoall_recv = (int *)malloc(size * size * sizeof(int));

    send_counts = (int *)malloc(size * sizeof(int));
    send_displs = (int *)malloc(size * sizeof(int));
    recv_counts = (int *)malloc(size * sizeof(int));
    recv_displs = (int *)malloc(size * sizeof(int));

    int send_index = 0;
    for (int i = 0; i < size; i++) {
        send_counts[i] = (rank + 1);
        send_displs[i] = send_index;
        send_index += send_counts[i];

        recv_counts[i] = (i + 1);
        recv_displs[i] = (i == 0) ? 0 : recv_displs[i - 1] + recv_counts[i - 1];
    }

    for (int i = 0; i < send_index; i++) {
        alltoall_send[i] = rank * 100 + i;
    }

    MPI_Alltoallv(alltoall_send, send_counts, send_displs, MPI_INT, alltoall_recv,
        recv_counts, recv_displs, MPI_INT, MPI_COMM_WORLD);

    MPI_Barrier(MPI_COMM_WORLD);  // Sync before gathering

    if (rank == ROOT) {
        gather_counts = (int *)malloc(size * sizeof(int));
        gather_displs = (int *)malloc(size * sizeof(int));

        int index = 0;
        for (int i = 0; i < size; i++) {
            gather_counts[i] = (i + 1) * 2;
            gather_displs[i] = index;
            index += gather_counts[i];
        }
    }

    // Gather all modified data
    MPI_Gatherv(recv_chunk, recv_size, MPI_INT, send_data, gather_counts, gather_displs,
        MPI_INT, ROOT, MPI_COMM_WORLD);

    if (rank == ROOT) {
        printf("Root gathered modified data: ");
        for (int i = 0; i < total_size; i++) {
            printf("%d ", send_data[i]);
        }
        printf("\n");
    }


    MPI_Finalize();
    return 0;
}
```

### 1.13 Code Snippet #13

**Run with 4 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define N 4  // Matrix size
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each process handles part of the matrix
    int rows_per_proc = N / size;
    int local_matrix[rows_per_proc][N];
    int vector[N];
    int local_result[rows_per_proc];
    int final_result[N];

    if (rank == 0) {
        int full_matrix[N][N] = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12},
            {13, 14, 15, 16}
        };
        int full_vector[N] = {1, 1, 1, 1};

        MPI_Scatter(full_matrix, rows_per_proc * N, MPI_INT, local_matrix, rows_per_proc * N
            , MPI_INT, 0, MPI_COMM_WORLD);

        MPI_Bcast(full_vector, N, MPI_INT, 0, MPI_COMM_WORLD);

        for (int i = 0; i < N; i++) {
            vector[i] = full_vector[i];
        }
    } else {
        MPI_Scatter(NULL, rows_per_proc * N, MPI_INT, local_matrix, rows_per_proc * N,
            MPI_INT, 0, MPI_COMM_WORLD);
        MPI_Bcast(vector, N, MPI_INT, 0, MPI_COMM_WORLD);
    }

    for (int i = 0; i < rows_per_proc; i++) {
        local_result[i] = 0;
        for (int j = 0; j < N; j++) {
            local_result[i] += local_matrix[i][j] * vector[j];
        }
    }

    MPI_Gather(local_result, rows_per_proc, MPI_INT, final_result, rows_per_proc, MPI_INT,
        0, MPI_COMM_WORLD);

    int total_sum;
    MPI_Allreduce(&local_result, &total_sum, rows_per_proc, MPI_INT, MPI_SUM, MPI_COMM_WORLD
        );

    if (rank == 0) {
        for (int i = 0; i < N; i++)
            printf("%d ", final_result[i]);
        printf("\n");
        printf("Root Got: %d\n", total_sum);
    }
    MPI_Finalize();
    return 0;
}
```

### 1.14 Code Snippet #14

**Run with 4 processes**

```c

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Ensure we have at least 3 processes
    if (size < 3) {
        fprintf(stderr, "Run with at least 3 processes!\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int send_value = rank + 1;  // Each process sends its rank + 1
    int *allgather_result = (int *)malloc(size * sizeof(int));

    MPI_Allgather(&send_value, 1, MPI_INT, allgather_result, 1, MPI_INT, MPI_COMM_WORLD);

    printf("Rank %d - Allgather result: ", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", allgather_result[i]);
    }
    printf("\n");

    int local_size = rank + 1;
    int *send_buffer = (int *)malloc(local_size * sizeof(int));
    for (int i = 0; i < local_size; i++)
        send_buffer[i] = rank * 10 + i;

    int *recv_counts = (int *)malloc(size * sizeof(int));
    MPI_Allgather(&local_size, 1, MPI_INT, recv_counts, 1, MPI_INT, MPI_COMM_WORLD);

    int *displacements = (int *)malloc(size * sizeof(int));
    displacements[0] = 0;
    int total_size = recv_counts[0];
    for (int i = 1; i < size; i++) {
        displacements[i] = displacements[i - 1] + recv_counts[i - 1];
        total_size += recv_counts[i];
    }

    int *allgatherv_result = (int *)malloc(total_size * sizeof(int));

    MPI_Allgatherv(send_buffer, local_size, MPI_INT, allgatherv_result, recv_counts,
        displacements, MPI_INT, MPI_COMM_WORLD);

    printf("Rank %d - Allgatherv result: ", rank);
    for (int i = 0; i < total_size; i++) {
        printf("%d ", allgatherv_result[i]);
    }
    printf("\n");

    int *send_alltoall = (int *)malloc(size * sizeof(int));
    int *recv_alltoall = (int *)malloc(size * sizeof(int));

    for (int i = 0; i < size; i++) send_alltoall[i] = rank * 100 + i;

    MPI_Alltoall(send_alltoall, 1, MPI_INT, recv_alltoall, 1, MPI_INT, MPI_COMM_WORLD);

    printf("Rank %d - Alltoall result: ", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", recv_alltoall[i]);
    }
```

```
    printf("\n");

    int *send_counts = (int *)malloc(size * sizeof(int));
    int *recv_counts_v = (int *)malloc(size * sizeof(int));
    int *send_displs = (int *)malloc(size * sizeof(int));
    int *recv_displs = (int *)malloc(size * sizeof(int));

    for (int i = 0; i < size; i++) {
        send_counts[i] = (rank + i) % size + 1;
        recv_counts_v[i] = (i + rank) % size + 1;
    }

    send_displs[0] = recv_displs[0] = 0;
    for (int i = 1; i < size; i++) {
        send_displs[i] = send_displs[i - 1] + send_counts[i - 1];
        recv_displs[i] = recv_displs[i - 1] + recv_counts_v[i - 1];
    }

    int send_total = send_displs[size - 1] + send_counts[size - 1];
    int recv_total = recv_displs[size - 1] + recv_counts_v[size - 1];

    int *send_data_v = (int *)malloc(send_total * sizeof(int));
    int *recv_data_v = (int *)malloc(recv_total * sizeof(int));

    for (int i = 0; i < send_total; i++) send_data_v[i] = rank * 10 + i;

    MPI_Alltoallv(send_data_v, send_counts, send_displs, MPI_INT, recv_data_v, recv_counts_v
        , recv_displs, MPI_INT, MPI_COMM_WORLD);

    printf("Rank %d - Alltoallv result: ", rank);
    for (int i = 0; i < recv_total; i++) {
        printf("%d ", recv_data_v[i]);
    }
    printf("\n");

    MPI_Finalize();
    return 0;
}
```

### 1.15 Code Snippet #15

**Run with 4 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial clues (rank 0 starts with 9)
    int clues = (rank == 0) ? 9 : 0;
    int fear = rank + 2; // Base fear level

    // 1. MPI_Scatter: Distribute clues
    int* clue_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
    if (rank == 0) {
        for (int i = 0; i < size; i++) clue_buf[i] = 2 + (i % 2); // 2 or 3
    }
    int local_clues;
    MPI_Scatter(clue_buf, 1, MPI_INT, &local_clues, 1, MPI_INT, 0, MPI_COMM_WORLD);
    clues = local_clues;
    fear += clues * 2; // Fear rises with clues

    // 2. MPI_Reduce: Total clues
    int total_clues = 0;
```

```
    MPI_Reduce(&clues, &total_clues, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // 3. MPI_Bcast: Share total clues
    MPI_Bcast(&total_clues, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 4. MPI_Alltoall: Trade fear levels
    int* fear_send = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) fear_send[i] = fear / (size - i);
    int* fear_recv = (int*)malloc(size * sizeof(int));
    MPI_Alltoall(fear_send, 1, MPI_INT, fear_recv, 1, MPI_INT, MPI_COMM_WORLD);
    for (int i = 0; i < size; i++) fear += fear_recv[i];

    // 5. MPI_Gather: Collect fear levels
    int* all_fears = (int*)malloc(size * sizeof(int));
    MPI_Gather(&fear, 1, MPI_INT, all_fears, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 6. MPI_Allreduce: Max fear
    int max_fear = 0;
    MPI_Allreduce(&fear, &max_fear, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    // Output
    printf("Rank %d: Clues %d, Fear %d, Max %d\n", rank, clues, fear, max_fear);
    if (rank == 0) printf("Total Clues: %d\n", total_clues);

    // Cleanup
    free(fear_send); free(fear_recv); free(all_fears);
    if (rank == 0) free(clue_buf);

    MPI_Finalize();
    return 0;
}
```

### 1.16 Code Snippet #16

**Run with 4 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial rumor value (starts at rank 0)
    int rumor = (rank == 0) ? 10 : 0; // Rumor starts as 10
    int heard = (rank == 0) ? 1 : 0; // 1 if heard, 0 if not

    // Step 1: Spread rumor locally (exaggerate if heard)
    if (heard) {
        rumor += rank * 2; // Exaggerate based on rank
    }

    // 2. MPI_Reduce: Count total who heard (to root)
    int total_heard = 0;
    MPI_Reduce(&heard, &total_heard, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // 3. MPI_Allreduce: Max rumor value across all
    int global_max_rumor = 0;
    MPI_Allreduce(&rumor, &global_max_rumor, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    // Step 4: Spread rumor to all via MPI_Alltoall
    int* send_rumor = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        send_rumor[i] = rumor + (i % 2 ? 1 : -1); // Slight variation per target
    }
    int* recv_rumor = (int*)malloc(size * sizeof(int));
```

```
    MPI_Alltoall(send_rumor, 1, MPI_INT, recv_rumor, 1, MPI_INT, MPI_COMM_WORLD);

    // Update rumor: Take max received value if not heard yet
    if (!heard) {
        rumor = recv_rumor[0];
        for (int i = 1; i < size; i++) {
            if (recv_rumor[i] > rumor) rumor = recv_rumor[i];
        }
        heard = 1;
    }

    // 5. MPI_Barrier: Sync before final output
    MPI_Barrier(MPI_COMM_WORLD);

    printf("Rank %d: Rumor %d, Heard %d, Global Max %d, Recv [",
           rank, rumor, heard, global_max_rumor);
    for (int i = 0; i < size; i++) printf("%d ", recv_rumor[i]);
    printf("]\n");
    if (rank == 0) printf("Total Heard: %d\n", total_heard);

    // Cleanup
    free(send_rumor); free(recv_rumor);

    MPI_Finalize();
    return 0;
}
```

### 1.17 Code Snippet #17

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial state: UFOs land at rank 0
    int ufos = (rank == 0) ? 3 : 0; // 3 UFOs start at rank 0
    int aliens = ufos * (rank + 1); // Aliens spawned: UFOs * (rank+1)

    // 1. MPI_Reduce: Total aliens to root
    int total_aliens = 0;
    MPI_Reduce(&aliens, &total_aliens, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // 2. MPI_Scatter: Distribute UFOs from root
    int* ufo_buffer = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
    if (rank == 0) {
        for (int i = 0; i < size; i++) ufo_buffer[i] = 1 + (i % 2); // 1 or 2 UFOs
    }

    int local_ufos;
    MPI_Scatter(ufo_buffer, 1, MPI_INT, &local_ufos, 1, MPI_INT, 0, MPI_COMM_WORLD);
    aliens += local_ufos * 2; // Each UFO spawns 2 more aliens

    // 3. MPI_Gather: Collect alien counts
    int* all_aliens = (int*)malloc(size * sizeof(int));
    MPI_Gather(&aliens, 1, MPI_INT, all_aliens, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 4. MPI_Alltoall: Spread aliens to neighbors
    int* send_aliens = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) send_aliens[i] = aliens / size + (i % 2);
    int* recv_aliens = (int*)malloc(size * sizeof(int));
    MPI_Alltoall(send_aliens, 1, MPI_INT, recv_aliens, 1, MPI_INT, MPI_COMM_WORLD);

```

```
    // Update aliens with received spread
    int new_aliens = aliens;
    for (int i = 0; i < size; i++) new_aliens += recv_aliens[i];

    // 5. MPI_Barrier: Sync before stats
    MPI_Barrier(MPI_COMM_WORLD);

    // 6. MPI_Allreduce: Max alien force
    int max_aliens = 0;
    MPI_Allreduce(&new_aliens, &max_aliens, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    // Output
    printf("Rank %d: UFOs %d, Aliens %d, Max Force %d\n",
           rank, local_ufos, new_aliens, max_aliens);
    if (rank == 0) {
        printf("Total Aliens: %d, All Counts [", total_aliens);
        for (int i = 0; i < size; i++) printf("%d ", all_aliens[i]);
        printf("]\n");
    }

    // Cleanup
    free(all_aliens); free(send_aliens); free(recv_aliens);
    if (rank == 0) free(ufo_buffer);

    MPI_Finalize();
    return 0;
}
```

### 1.18 Code Snippet #18

**Run with 4 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial swords (rank 0 starts with 10)
    int swords = (rank == 0) ? 10 : 0;
    int strength = rank + 3; // Base strength

    // 1. MPI_Scatter: Distribute swords
    int* sword_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
    if (rank == 0) {
        for (int i = 0; i < size; i++) sword_buf[i] = 3 - (i % 2); // 3 or 2
    }
    int local_swords;
    MPI_Scatter(sword_buf, 1, MPI_INT, &local_swords, 1, MPI_INT, 0, MPI_COMM_WORLD);
    swords = local_swords;
    strength += swords; // Strength from swords

    // 2. MPI_Reduce: Total dragons slain (swords = dragons)
    int total_dragons = 0;
    MPI_Reduce(&swords, &total_dragons, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // 3. MPI_Bcast: Share total dragons
    MPI_Bcast(&total_dragons, 1, MPI_INT, 0, MPI_COMM_WORLD);
    strength += total_dragons / size; // Boost from fame

    // 4. MPI_Alltoall: Trade dragon scales
    int* scale_send = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) scale_send[i] = strength / (i + 2);
    int* scale_recv = (int*)malloc(size * sizeof(int));
    MPI_Alltoall(scale_send, 1, MPI_INT, scale_recv, 1, MPI_INT, MPI_COMM_WORLD);
```

```
    for (int i = 0; i < size; i++) strength += scale_recv[i];

    // 5. MPI_Gather: Collect strengths
    int* all_strengths = (int*)malloc(size * sizeof(int));
    MPI_Gather(&strength, 1, MPI_INT, all_strengths, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 6. MPI_Allreduce: Max strength
    int max_strength = 0;
    MPI_Allreduce(&strength, &max_strength, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    // Output
    printf("Rank %d: Swords %d, Strength %d, Max %d\n", rank, swords, strength, max_strength
        );
    if (rank == 0) printf("Total Dragons: %d\n", total_dragons);

    // Cleanup
    free(scale_send); free(scale_recv); free(all_strengths);
    if (rank == 0) free(sword_buf);

    MPI_Finalize();
    return 0;
}
```

### 1.19 Code Snippet #19

**Run with 3 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial fuel (rank 0 starts with 12)
    int fuel = (rank == 0) ? 12 : 0;
    int speed = rank + 1; // Base speed per ship

    // 1. MPI_Scatter: Distribute fuel
    int* fuel_buf = (rank == 0) ? (int*)malloc(size * sizeof(int)) : NULL;
    if (rank == 0) {
        for (int i = 0; i < size; i++) fuel_buf[i] = 3 + (i % 2); // 3 or 4
    }
    int local_fuel;
    MPI_Scatter(fuel_buf, 1, MPI_INT, &local_fuel, 1, MPI_INT, 0, MPI_COMM_WORLD);
    fuel = local_fuel;
    speed += fuel; // Speed increases with fuel

    // 2. MPI_Bcast: Share total fuel
    int total_fuel = fuel;
    MPI_Reduce(&fuel, &total_fuel, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Bcast(&total_fuel, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 3. MPI_Alltoall: Trade speed boosts
    int* trade_send = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) trade_send[i] = speed / (i + 1);
    int* trade_recv = (int*)malloc(size * sizeof(int));
    MPI_Alltoall(trade_send, 1, MPI_INT, trade_recv, 1, MPI_INT, MPI_COMM_WORLD);
    for (int i = 0; i < size; i++) speed += trade_recv[i];

    // 4. MPI_Gather: Collect speeds
    int* all_speeds = (int*)malloc(size * sizeof(int));
    MPI_Gather(&speed, 1, MPI_INT, all_speeds, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // 5. MPI_Allreduce: Max speed
    int max_speed = 0;
```

```
    MPI_Allreduce(&speed, &max_speed, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    // Output
    printf("Rank %d: Fuel %d, Speed %d, Max %d\n", rank, fuel, speed, max_speed);
    if (rank == 0) printf("Total Fuel: %d\n", total_fuel);

    // Cleanup
    free(trade_send); free(trade_recv); free(all_speeds);
    if (rank == 0) free(fuel_buf);

    MPI_Finalize();
    return 0;
}
```

### 1.20 Code Snippet #20

**Run with 4 processes**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initial treasures (rank 0 starts with 5)
    int treasures = (rank == 0) ? 5 : 0;

    // Rank 0 sends treasures to rank 1
    if (rank == 0) {
        MPI_Send(&treasures, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    if (rank == 1) {
        MPI_Recv(&treasures, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        treasures += 1; // Rank 1 adds a find
    }

    // Allocate memory for sendcounts and displs
    int* sendcounts = (int*)malloc(size * sizeof(int));
    int* displs = (int*)malloc(size * sizeof(int));
    int total = 0;

    // Initialize sendcounts and displs on rank 0
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            sendcounts[i] = 1 + (i % 2); // 1 or 2 treasures
            displs[i] = total;
            total += sendcounts[i];
        }
    }

    // Broadcast sendcounts and displs to all ranks
    MPI_Bcast(sendcounts, size, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(displs, size, MPI_INT, 0, MPI_COMM_WORLD);

    // Allocate memory for local_treasures
    int* local_treasures = (int*)malloc(sendcounts[rank] * sizeof(int));

    if (rank == 0) {
        int* treasures_array = (int*)malloc(total * sizeof(int));
        for (int i = 0; i < total; i++) {
            treasures_array[i] = treasures; // Fill with initial treasures
        }
        MPI_Scatterv(treasures_array, sendcounts, displs, MPI_INT, local_treasures,
            sendcounts[rank], MPI_INT, 0, MPI_COMM_WORLD);
        free(treasures_array);
```

```
    } else {
        MPI_Scatterv(NULL, sendcounts, displs, MPI_INT, local_treasures, sendcounts[rank],
            MPI_INT, 0, MPI_COMM_WORLD);
    }

    for (int i = 0; i < sendcounts[rank]; i++) {
        treasures += local_treasures[i];
    }

    // Reduce total treasures
    int total_treasures = 0;
    MPI_Reduce(&treasures, &total_treasures, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    int* trade_send = (int*)malloc(size * sizeof(int));
    int* trade_recv = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        trade_send[i] = treasures / (size - i);
    }
    MPI_Alltoall(trade_send, 1, MPI_INT, trade_recv, 1, MPI_INT, MPI_COMM_WORLD);

    for (int i = 0; i < size; i++) {
        treasures += trade_recv[i];
    }

    int* all_treasures = (int*)malloc(total * sizeof(int));
    MPI_Gatherv(&treasures, sendcounts[rank], MPI_INT, all_treasures, sendcounts, displs,
        MPI_INT, 0, MPI_COMM_WORLD);

    int max_treasures = 0;
    MPI_Allreduce(&treasures, &max_treasures, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

    printf("Rank %d: Treasures %d, Max %d\n", rank, treasures, max_treasures);
    if (rank == 0) {
        printf("Total Treasures: %d, [", total_treasures);
        for (int i = 0; i < total; i++) {
            printf("%d ", all_treasures[i]);
        }
        printf("]\n");
    }

    MPI_Finalize();
    return 0;
}
```

## 2 Expected Outputs

Below are the expected outputs for each code snippet, presented in a formatted manner for clarity.

### Output for Code Snippet #1

```
Both of the data send
Data received in rank 1, data = 7, -1
```

### Output for Code Snippet #2

```
Both of the data send
Count of data received = 0
Data received in rank 1, data = -1, -1
```

**Output for Code Snippet #3**

```
Process 1 :
1 3 5
Process 2 :
2 5 8
Process 0 :
0 1 2
Process 0 received: 0 1 2
Process 1 received: 1 3 5
Process 2 received: 2 5 8
```

**Output for Code Snippet #4**

```
Master sent task 0 to worker 1
Master sent task 1 to worker 2
Master sent task 2 to worker 1
Master sent task 3 to worker 2
Master sent task 4 to worker 1
Worker 2 received task: 4 5 6
Master received result from worker 2: 8 10 12
Worker 2 finished sending result.
Worker 2 received task: 10 11 12
Master received result from worker 2: 20 22 24
Worker 2 finished sending result.
Worker 1 received task: 1 2 3
Worker 1 finished sending result.
Worker 1 received task: 7 8 9
Worker 1 finished sending result.
Worker 1 received task: 13 14 15
Worker 1 finished sending result.
Master received result from worker 1: 2 4 6
Master received result from worker 1: 14 16 18
Master received result from worker 1: 26 28 30
Master sent termination message to worker 1
Master sent termination message to worker 2
Worker 2 received termination signal. Exiting.
Worker 1 received termination signal. Exiting.
```

**Output for Code Snippet #5**

```
Both of the data send
Data received in rank 1, data = 7, 7
```

**Output for Code Snippet #6**

```
Both of the data send
Data received in rank 1, data = 7, -1
```

**Output for Code Snippet #7**

```
Both of the data send
// Error
MPI_Recv(buf=0x7ffccaf73790, count=1, MPI_INT, src=0, tag=0, MPI_COMM_WORLD, status=0x1)
failed MPIDI_CH3_PktHandler_EagerShortSend(363): Message from rank 0 and tag 0 truncated;
8 bytes received but buffer size is 4
```

**Output for Code Snippet #8**

```
Root initialized data1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Root initialized data2: 3 6 9 12 15 18 21 24 27 30 33 36 39 42 45 48
Root gathered modified data1: 1 2 3 4 5 6 7 8 9 10 22 24 26 28 30 16
Process 0 received value 130 from process 2
Process 1 received value 15 from process 0
Process 2 received value 40 from process 1
```

**Output for Code Snippet #9**

```
Process 0 received config_value = 5
Process 1 received config_value = 5
Process 4 received config_value = 5
Process 5 received config_value = 5
Process 2 received config_value = 5
Process 3 received config_value = 5
Final result gathered by root process:
5 10 15 20 25 30 35 40 45 50 55 60
```

**Output for Code Snippet #10**

```
Root initialized data: 1 2 3 4 5 6
Root gathered modified data: 2 6 9 16 20 24
```

**Output for Code Snippet #11**

```
Final result gathered by root process:
1 2 3 4 10 12 14 16 27 30 33 36
Process 0 received: 1 2 3
Process 2 received: 1 2 3
Process 1 received: 1 2 3
```

**Output for Code Snippet #12**

```
Root initialized data: 1 2 3 4 5 6
Root gathered modified data: 3 6 18 24 30 36
```

**Output for Code Snippet #13**

```
10 26 42 58
Root Got: 136
```

**Output for Code Snippet #14**

```
Rank 0 - Allgather result: 1 2 3 4
Rank 1 - Allgather result: 1 2 3 4
Rank 2 - Allgather result: 1 2 3 4
Rank 3 - Allgather result: 1 2 3 4
Rank 0 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 3 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 2 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 1 - Allgatherv result: 0 10 11 20 21 22 30 31 32 33
Rank 1 - Alltoall result: 1 101 201 301
Rank 2 - Alltoall result: 2 102 202 302
Rank 0 - Alltoall result: 0 100 200 300
Rank 3 - Alltoall result: 3 103 203 303
Rank 3 - Alltoallv result: 6 7 8 9 19 28 29 37 38 39
Rank 0 - Alltoallv result: 0 10 11 20 21 22 30 31 32 33
Rank 2 - Alltoallv result: 3 4 5 15 16 17 18 27 35 36
Rank 1 - Alltoallv result: 1 2 12 13 14 23 24 25 26 34
```

**Output for Code Snippet #15**

```
Rank 0: Clues 2, Fear 13, Max 45
Rank 1: Clues 3, Fear 19, Max 45
Rank 2: Clues 2, Fear 24, Max 45
Rank 3: Clues 3, Fear 45, Max 45
Total Clues: 10
```

**Output for Code Snippet #16**

```
Rank 0: Rumor 10, Heard 1, Global Max 10, Recv [9 -1 -1 -1 ]
Rank 1: Rumor 11, Heard 1, Global Max 10, Recv [11 1 1 1 ]
Rank 2: Rumor 9, Heard 1, Global Max 10, Recv [9 -1 -1 -1 ]
Rank 3: Rumor 11, Heard 1, Global Max 10, Recv [11 1 1 1 ]
Total Heard: 1
```

**Output for Code Snippet #17**

```
Rank 0: UFOs 1, Aliens 7, Max Force 9
Rank 1: UFOs 2, Aliens 9, Max Force 9
Rank 2: UFOs 1, Aliens 4, Max Force 9
Total Aliens: 3, All Counts [5 4 2 ]
```

**Output for Code Snippet #18**

```
Rank 0: Swords 3, Strength 26, Max 26
Rank 2: Swords 3, Strength 18, Max 26
Rank 1: Swords 2, Strength 18, Max 26
Rank 3: Swords 2, Strength 16, Max 26

Total Dragons: 10
```

**Output for Code Snippet #19**

```
Rank 0: Fuel 3, Speed 20, Max 20
Rank 2: Fuel 3, Speed 11, Max 20
Rank 1: Fuel 4, Speed 14, Max 20
Total Fuel: 10
```

**Output for Code Snippet #20**

```
Rank 0: Treasures 19, Max 51
Rank 1: Treasures 28, Max 51
Rank 2: Treasures 25, Max 51
Rank 3: Treasures 51, Max 51
Total Treasures: 41, [19 28 0 25 51 0]
```